Great Place To Work. Certified FEB 2023 – FEB 2024 INDIA

UPES

UNIVERSITY OF TOMORROW

nirf RANKING 2025

*RANKED 45 IN INDIA

#University Category

QS WORLD UNIVERSITY RANKINGS

NO.1 PVT. UNIVERSITY IN ACADEMIC REPUTATION IN INDIA

NAAC

ACCREDITED **GRADE 'A'** BY NAAC

PERFECT SCORE OF **150/150** AS A TESTAMENT TO EXECEPTIONAL E-LEARNING METHODS

UPES UNIVERSITY OF TOMORROW

Applied Machine Learning

Unit 07 – Lecture 21: Similarity Measures and Data Types in Clustering

Tofik Ali
School of Computer Science, UPES Dehradun

Autumn 2026

Every clustering algorithm takes a distance as input

Topic focus: **get the distance right before you run anything**

Repository: `https://github.com/tali7c/Applied-Machine-Learning`

Sources: Han, Kamber and Pei, *Data Mining: Concepts and Techniques* (3e), Ch. 10.;James et al., *An Introduction to Statistical Learning with Python*, Ch. 12.;Müller and Guido, *Introduction to Machine Learning with Python*, §3.5.;Ester, Kriegel, Sander and Xu, "A density-based algorithm for discovering clusters (DBSCAN)," KDD 1996.

Quick Links

What a distance must be
Minkowski family
Cosine vs Euclidean
Binary attributes
Nominal and ordinal
Mixed types
Standardisation
Centroids and linkage
Summary

Agenda

What a distance has to be

Numeric attributes: the Minkowski family

Cosine similarity against Euclidean distance

Binary attributes

Nominal and ordinal attributes

Mixed attribute types

Standardisation changes the answer

Cluster centroid, medoid and cluster distances

Does any of this actually change a clustering?

How this lecture works

Every idea appears twice

1. **By hand** — two points, three patients, four applicants, six points on a page. Small enough to check with a pen.
2. **In code** — the same arithmetic again inside `scipy` and `scikit-learn`, so you can see the library is doing exactly this and nothing magic.

Commit before you look

Five slides ask you to predict a number, or a winner, before it is revealed. Write your guess down. Being wrong on paper is how the idea sticks.

This is the most **numerical** lecture in the unit. The binary contingency table and the mixed-type table are the two calculations you should be able to reproduce with a pen, and both are examinable.

One seed, and three places it is used

The data

Bundled `load_wine`, `load_iris`, `load_breast_cancer`, and small tables typed out in the code. Nothing is downloaded and nothing is generated.

The randomness

One seed for the whole lecture: `np.random.default_rng(0)`, also `random_state=0`. The 200 axiom points, the 400 ordering pairs, one `KMeans`. Every listing shows it.

Why this matters to you

Every number in this deck is printed by one script. Run the code as written and you get the same numbers — which is the only reason to believe a number on a slide.

Where we are

The previous lecture defined the problem

Clustering groups objects so that objects in the same group are *similar* and objects in different groups are not. No labels, no supervision, no test set.

It left one word undefined

Similar. Every algorithm in this unit — hierarchical, k-means, k-medoids, DBSCAN — takes a distance as its input and is only as good as that distance.

Today's one question

Given two rows of a table, how far apart are they? *And what if the columns are not numbers?*

Why it comes before the algorithms

You cannot debug a clustering by staring at the algorithm. If the answer is wrong, the distance is usually why. Fixing a distance is cheap; re-running a bad one on a bigger machine is not.

Two words, used carefully

Similarity $s(i, j)$

Large when the objects are alike. Usually scaled to $[0, 1]$, with $s(i, i) = 1$. Cosine similarity, the Jaccard coefficient and the simple matching coefficient are all similarities.

Dissimilarity $d(i, j)$

Small when the objects are alike, $d(i, i) = 0$. Euclidean distance is a dissimilarity. So is $1 - s$ whenever s lives in $[0, 1]$.

The object every algorithm actually consumes

The **dissimilarity matrix** D, with $D_{ij} = d(i, j)$: an $n \times n$ symmetric table with a zero diagonal.

Not the data. The *distances*. That is the whole interface.

Convert freely: $d = 1 - s$, or $d = \sqrt{2(1 - s)}$, or $s = 1/(1 + d)$. But convert *deliberately* — the choice changes the geometry, as we will see.

Four axioms make a metric

1. **Non-negativity** $d(i,j) \geq 0$
2. **Identity of indiscernibles** $d(i,j) = 0 \iff i = j$
3. **Symmetry** $d(i,j) = d(j,i)$
4. **Triangle inequality** $d(i,k) \leq d(i,j) + d(j,k)$

A dissimilarity satisfying all four is a **metric**.

Why they are not decoration

(3) A clustering must not depend on the order you read the rows.

(4) No detour is shorter than the direct route — this is what lets k-d trees, ball trees and the triangle-inequality pruning in k-means skip comparisons. Break it and those optimisations return wrong answers, silently.

The axioms, checked

```
rng = np.random.default_rng(0) # one seed for the whole lecture
D = squareform(pdist(rng.normal(size=(200, 4)), "euclidean"))
print(D[off].min(), np.abs(np.diag(D)).max(), np.abs(D - D.T).max())
worst = max(D[i,k] - (D[i,j] + D[j,k]) for i,j,k in combinations(range
    (60),3))
```

```
non-negativity   min off-diagonal d = 0.169507  (>= 0)
identity         max |d(i,i)|        = 0.000000  (== 0)
identity         any d(i,j)==0 for i!=j : False
symmetry         max |d(i,j)-d(j,i)| = 0.000e+00
triangle         max d(i,k)-(d(i,j)+d(j,k)) = -0.007349  (<= 0)
```

The triangle number is the interesting one: over 34 220 triples the *worst* slack is -0.007349, so it is never violated, but it comes close — nearly-collinear triples.

By hand: two named measures that fail

Squared Euclidean distance

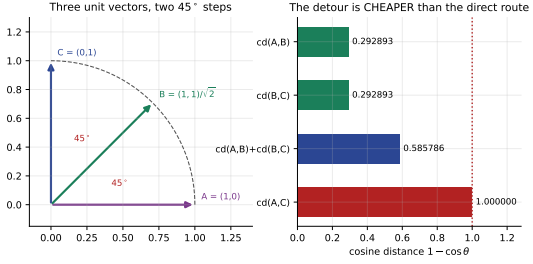
$a = (0,0),\ b = (1,0),\ c = (2,0).$

$$d^2(a,b) = 1,\ d^2(b,c) = 1,\ d^2(a,c) = 4$$

$$4 \ > \ 1 + 1 = 2$$

The triangle inequality **fails** — and this is exactly what k-means minimises. A *divergence*, not a metric.

Squaring is monotone, so the *ranking* of pairs is unchanged. What breaks is the arithmetic *between* distances.



Cosine distance fails too: two 45° steps cost 0.292893 each, the one 90° step costs 1. **The detour is cheaper than the direct route.** The repair is the *angular* distance $\arccos(\cos\theta)/\pi$: $0.25 + 0.25 = 0.5$ exactly.

And one that is not even symmetric

```
kl = lambda u, v: np.sum(u * np.log2(u / v))
p, q = np.array([0.5, 0.3, 0.2]), np.array([0.2, 0.4, 0.4])
```

```
KL(p||q) = 0.336453   KL(q||p) = 0.301629   difference 0.034823
```

measure	d>=0	d(i,i)=0	symmetry	triangle
Euclidean	yes	yes	yes	yes
Manhattan	yes	yes	yes	yes
Minkowski p>=1	yes	yes	yes	yes
Chebyshev	yes	yes	yes	yes
squared Euclidean	yes	yes	yes	NO
cosine distance	yes	no*	yes	NO
Jaccard distance	yes	yes	yes	yes
simple matching dist	yes	yes	yes	yes
KL divergence	yes	yes	NO	NO

$*$ cosine distance is 0 for any two *parallel* vectors, not only equal ones:
$\mathrm{cd}((1,2),(2,4)) = 0.000000$.

One formula, one dial

$$d_p(x, y) = \left(\sum_{f=1}^{m} |x_f - y_f|^p \right)^{1/p}, \qquad p \geq 1$$

$p = 1$

Manhattan, city-block, L_1. Add the gaps up.

$p = 2$

Euclidean, L_2. The straight line. The default everywhere.

$p \to \infty$

Chebyshev, supremum, L_∞. Only the single largest gap survives.

p controls *how much a single bad attribute is punished*. Large p: one large disagreement dominates. $p = 1$: it does not.

By hand: one pair, four distances

$x = (1, 2)$, $y = (4, 6)$, componentwise gaps 3 and 4

$$d_1 = |3| + |4| = \mathbf{7}$$

$$d_2 = \sqrt{3^2 + 4^2} = \sqrt{9 + 16} = \sqrt{25} = \mathbf{5} \quad \text{(the 3–4–5 triangle)}$$

$$d_3 = (3^3 + 4^3)^{1/3} = (27 + 64)^{1/3} = 91^{1/3} = \mathbf{4.497941}$$

$$d_\infty = \max(3, 4) = \mathbf{4}$$

Notice the order: $7 > 5 > 4.498 > 4$. That is general — $d_1 \geq d_2 \geq d_3 \geq \cdots \geq d_\infty$ for every pair of points, always. Verified on 400 random pairs in $\mathbb{R}^5$: all three inequalities hold in 400/400 cases.

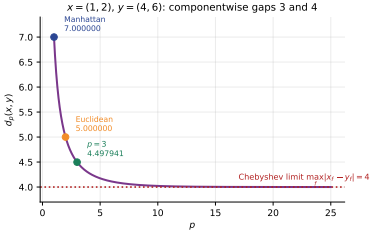
The same thing in code

```
from scipy.spatial.distance import cityblock, euclidean, minkowski,
    chebyshev
x, y = np.array([1., 2.]), np.array([4., 6.])
for p in [1, 1.5, 2, 3, 4, 6, 8, 12, 20, 50, 100]:
    print(p, minkowski(x, y, p=p))
```

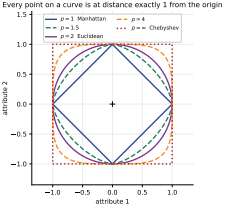
```
     p      d_p(x,y)
     1      7.000000         6      4.110704
   1.5      5.584250         8      4.047992
     2      5.000000        12      4.010409
     3      4.497941        20      4.000633
     4      4.284572        50      4.000000
                           inf      4.000000
```

The limit is $\max_f |x_f - y_f| = 4$, reached to six decimal places by $p = 50$. Chebyshev is not a separate idea; it is the end of the dial.

Drawn: what the dial actually does



Strictly decreasing in p, flattening onto the Chebyshev limit. Nothing is special about $p = 2$ except habit.



Every point on a curve is at distance **exactly 1** from the centre. Along the diagonal the two attributes disagree equally: $p = 1$ pulls that direction in tightest, $p = \infty$ pushes it furthest out.

Large p forgives spreading a disagreement across attributes; $p = 1$ does not care how it is spread.

Predict first #1 — which point is nearer?

Query $q = (0, 0)$. Candidate $u = (3, 3)$ — disagrees a little on both attributes. Candidate $v = (4.5, 0)$ — disagrees a lot on one.

Which is q's nearest neighbour? Commit before you look.

Predict first #1 — which point is nearer?

Query $q = (0, 0)$. Candidate $u = (3, 3)$ — disagrees a little on both attributes. Candidate $v = (4.5, 0)$ — disagrees a lot on one.

Which is q's nearest neighbour? Commit before you look.

metric	d(q,u)	d(q,v)	nearest
Manhattan	6.000000	4.500000	v
Euclidean	4.242641	4.500000	u
Minkowski3	3.779763	4.500000	u
Chebyshev	3.000000	4.500000	u

Predict first #1 — which point is nearer?

Query $q = (0, 0)$. Candidate $u = (3, 3)$ — disagrees a little on both attributes. Candidate $v = (4.5, 0)$ — disagrees a lot on one.

Which is q's nearest neighbour? Commit before you look.

metric	d(q,u)	d(q,v)	nearest
Manhattan	6.000000	4.500000	v
Euclidean	4.242641	4.500000	u
Minkowski3	3.779763	4.500000	u
Chebyshev	3.000000	4.500000	u

The answer is: it depends, and that is the point

Same data, same query, **different neighbour**. p is not a tuning detail — it encodes what you mean by "alike". If a customer who is a bit unusual on every attribute should count as more normal than one who is wildly unusual on a single attribute, you want large p.

By hand: the pair that breaks the tie

$\cos(x, y) = \dfrac{x \cdot y}{\|x\| \, \|y\|}$ — the cosine of the angle. Range $[0, 1]$ for count data.

$\cos(x, kx) = 1$ for every $k > 0$: **length is discarded**.

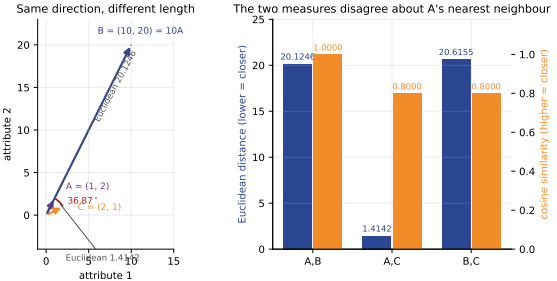
$A = (1, 2), \quad B = (10, 20) = 10A, \quad C = (2, 1)$

$$d(A, B) = \sqrt{9^2 + 18^2} = \sqrt{405} = 20.124612 \qquad d(A, C) = \sqrt{1^2 + 1^2} = \sqrt{2} = 1.414214$$

$$\cos(A, B) = \frac{10 + 40}{\sqrt{5} \cdot \sqrt{500}} = \frac{50}{50} = 1.000000 \qquad \cos(A, C) = \frac{2 + 2}{\sqrt{5} \cdot \sqrt{5}} = \frac{4}{5} = 0.800000$$

Euclidean says C is A's nearest neighbour, by a factor of fourteen. **Cosine** says B *is* A, and C is 36.87° away. Neither is wrong; they answer different questions.

Drawn: the disagreement



The bars point in opposite directions: for Euclidean, lower is closer; for cosine, higher is closer. A, B is the best possible cosine score and the second-worst Euclidean score in the same picture.

If magnitude carries information you have just discarded it; if magnitude is a nuisance, you have just removed it on purpose.

Predict first #2 — three documents

$$\texttt{d1} = [1, 2, 1, 0, 4] \quad \texttt{d2} = [10, 20, 10, 0, 40] \quad \texttt{d3} = [0, 1, 0, 3, 4]$$

`d2` is `d1` with every count multiplied by ten — the same note, ten times as long. `d3` is a different note of the same length as `d1`.

Under Euclidean distance, which pair is closest?

Predict first #2 — three documents

$$\texttt{d1} = [1, 2, 1, 0, 4] \quad \texttt{d2} = [10, 20, 10, 0, 40] \quad \texttt{d3} = [0, 1, 0, 3, 4]$$

d2 is d1 with every count multiplied by ten — the same note, ten times as long. d3 is a different note of the same length as d1.

Under Euclidean distance, which pair is closest?

doc	doc	euclid	cosine
d1 short note	d2 long paper	42.2137	1.0000
d1 short note	d3 short note	3.4641	0.7526
d2 long paper	d3 short note	43.1972	0.7526

Predict first #2 — three documents

$$\texttt{d1} = [1, 2, 1, 0, 4] \quad \texttt{d2} = [10, 20, 10, 0, 40] \quad \texttt{d3} = [0, 1, 0, 3, 4]$$

`d2` is `d1` with every count multiplied by ten — the same note, ten times as long. `d3` is a different note of the same length as `d1`.

Under Euclidean distance, which pair is closest?

doc	doc	euclid	cosine
d1 short note	d2 long paper	42.2137	1.0000
d1 short note	d3 short note	3.4641	0.7526
d2 long paper	d3 short note	43.1972	0.7526

Euclidean pairs the two short documents

`d1` and `d3` are 3.46 apart because they are both *short*, not because they are about the same thing. `d1` and `d2` — literally the same document — are 42.21 apart. On text, Euclidean distance clusters by length. **This is why text is the canonical cosine application.**

The bridge: normalise the rows and they are the same

$$\left\| \frac{a}{\|a\|} - \frac{b}{\|b\|} \right\|^2 = 2\big(1 - \cos(a, b)\big)$$

```
uh, vh = u / np.linalg.norm(u), v / np.linalg.norm(v)
print(np.sum((uh - vh)**2), 2 * (1 - cosine_similarity([u], [v])[0, 0]))
```

```
A,B    ||ahat-bhat||^2 = 0.0000000000   2(1-cos) = 0.0000000000   diff 2.22e-16
A,C    ||ahat-bhat||^2 = 0.4000000000   2(1-cos) = 0.4000000000   diff 3.89e-16
B,C    ||ahat-bhat||^2 = 0.4000000000   2(1-cos) = 0.4000000000   diff 1.67e-16
```

So "use cosine" and "L_2-normalise every row, then use Euclidean" are **the same procedure**. That is often the easier one to implement, because every library takes Euclidean.

The 2×2 contingency table

		object j		
		1	0	sum
object i	1	q	r	$q + r$
	0	s	t	$s + t$
	sum	$q + s$	$r + t$	p

$p = q + r + s + t$ is the number of attributes.

The four counts

q — both 1 (a shared *presence*)
r — i has it, j does not
s — j has it, i does not
t — both 0 (a shared *absence*)

One question decides everything

Is t evidence of similarity?
Two patients who both tested negative for forty rare diseases — are they alike?

Two answers, two coefficients

Symmetric attributes

Both outcomes equally informative — `gender`, coin flips, `is_urban`.

$$\mathrm{SMC}(i,j) = \frac{q+t}{q+r+s+t}$$

$$d(i,j) = \frac{r+s}{q+r+s+t}$$

The **simple matching coefficient**. Counts t.

Asymmetric attributes

One outcome rare and informative — a positive test, a purchase, a word occurring.

$$J(i,j) = \frac{q}{q+r+s}$$

$$d(i,j) = \frac{r+s}{q+r+s}$$

The **Jaccard coefficient**. Ignores t entirely.

The only difference is whether t appears in the denominator. That one term is the whole examinable distinction.

By hand: three patients, six binary tests

	fever	cough	test1	test2	test3	test4
Jack	1	0	1	0	0	0
Jim	1	1	0	0	0	0
Mary	1	0	1	0	1	0

Jack against Mary — count, then divide

Both 1: `fever`, `test1` $\Rightarrow q = 2$. Jack 1, Mary 0: none $\Rightarrow r = 0$. Mary 1, Jack 0: `test3` $\Rightarrow s = 1$. Both 0: `cough`, `test2`, `test4` $\Rightarrow t = 3$.

$$\text{SMC} = \frac{2+3}{6} = 0.8333 \qquad J = \frac{2}{2+0+1} = \frac{2}{3} = 0.6667$$

All three pairs, machine-checked

```
q = np.sum((u == 1) & (v == 1)); r = np.sum((u == 1) & (v == 0))
s = np.sum((u == 0) & (v == 1)); t = np.sum((u == 0) & (v == 0))
smc = (q + t) / (q + r + s + t); jac = q / (q + r + s)
```

```
pair          q   r   s   t |   SMC sim   SMC dis |   Jac sim   Jac dis
Jack,Jim      1   1   1   3 |    0.6667    0.3333 |    0.3333    0.6667
Jack,Mary     2   0   1   3 |    0.8333    0.1667 |    0.6667    0.3333
Jim,Mary      1   1   2   2 |    0.5000    0.5000 |    0.2500    0.7500
```

```
Jack,Jim     scipy hamming 0.333333   scipy jaccard 0.666667
Jack,Mary    scipy hamming 0.166667   scipy jaccard 0.333333
Jim,Mary     scipy hamming 0.500000   scipy jaccard 0.750000
```

For 0/1 vectors `scipy`'s `hamming` — the proportion of positions that disagree — **is** the simple-matching dissimilarity $(r + s)/p$.

Why the choice matters: pad with shared zeros

```
u = 1010, v = 1100, then
append k attributes that
are 0 for BOTH objects:
      k    SMC  Jaccard
      0  0.5000   0.3333
      4  0.7500   0.3333
     16  0.9000   0.3333
     96  0.9800   0.3333
    996  0.9980   0.3333
```

The market-basket argument

Add a thousand products **nobody bought** to the catalogue. Two shoppers who bought different things are now 99.8% similar under simple matching, and unchanged under Jaccard. Adding items nobody bought cannot make two shoppers more alike.

Predict first #3 — the two coefficients cross

pair 1	$u = 1100000000$	$q = 1, r = 1, s = 1, t = 7$
	$v = 1010000000$	
pair 2	$u = 1111111100$	$q = 6, r = 2, s = 2, t = 0$
	$v = 1111110011$	

Simple matching makes pair 1 the more similar (0.8000 **vs** 0.6000**). What does Jaccard say?**

Predict first #3 — the two coefficients cross

pair 1 $u = 1100000000$ $q = 1, r = 1, s = 1, t = 7$
$v = 1010000000$

pair 2 $u = 1111111100$ $q = 6, r = 2, s = 2, t = 0$
$v = 1111110011$

Simple matching makes pair 1 the more similar (0.8000 vs 0.6000). What does Jaccard say?

```
simple matching: pair 1 (0.8000) is more similar than pair 2 (0.6000)
Jaccard:         pair 2 (0.6000) is more similar than pair 1 (0.3333)
```

Predict first #3 — the two coefficients cross

pair 1 $u = 1100000000$ $q = 1, r = 1, s = 1, t = 7$
$v = 1010000000$

pair 2 $u = 1111111100$ $q = 6, r = 2, s = 2, t = 0$
$v = 1111110011$

Simple matching makes pair 1 the more similar (0.8000 vs 0.6000). What does Jaccard say?

```
simple matching: pair 1 (0.8000) is more similar than pair 2 (0.6000)
Jaccard:         pair 2 (0.6000) is more similar than pair 1 (0.3333)
```

Opposite rankings on the same data

Not a rounding difference — a **reversal**. Any clustering built on these two coefficients will put different patients together. The decision "is a shared absence evidence?" is made once, before any algorithm runs, and it decides the answer.

The $0/1$ coding is not a free choice

```
u = [1 1 1 0 0]   v = [1 0 0 0 0]
   q=1 r=2 s=0 t=2 ->  Jaccard sim 0.3333    SMC sim 0.6000
now swap the meaning of 0 and 1 on every attribute:
u = [0 0 0 1 1]   v = [0 1 1 1 1]
   q=2 r=0 s=2 t=1 ->  Jaccard sim 0.5000    SMC sim 0.6000
```

Simple matching: unchanged

0.6000 either way. It is symmetric in the two codes — which is exactly what "symmetric attribute" means.

Jaccard: $0.3333 \rightarrow 0.5000$

It counts only whichever value you coded as 1. **Decide which outcome is the rare, informative one before you compute anything.**

Nominal: categories with no order

$$d(i,j) = \frac{p - m}{p}$$

p = number of nominal attributes, m = number on which i and j take the *same* value.
The **mismatch ratio**.

Three objects, three nominal attributes

	colour	shape	material
A	red	round	wood
B	red	square	wood
C	blue	square	metal

pair	p	m	$(p-m)/p$
A,B	3	2	$1/3 = 0.333333$
A,C	3	0	$3/3 = 1.000000$
B,C	3	1	$2/3 = 0.666667$

The one-hot alternative — equivalent, or not?

```
columns: colour=blue, colour=red, shape=round, shape=square, material=metal, material=wood
  A        0  1  1  0  0  1
  B        0  1  0  1  0  1
  C        1  0  0  1  1  0

pair        (p-m)/p      L1/(2p)   L1 one-hot   L2 one-hot
A,B        0.333333     0.333333     2.000000     1.414214
A,C        1.000000     1.000000     6.000000     2.449490
B,C        0.666667     0.666667     4.000000     2.000000
```

L_1: exactly the same measure

Each mismatch flips two dummy columns, so $L_1 = 2(p - m)$ and $L_1/(2p) = (p - m)/p$ to the last digit.

L_2: same ranking, different numbers

$L_2 = \sqrt{2(p - m)}$. The extreme ratio is 3.0000 for the mismatch ratio but only 1.7321 for L_2 — **Euclidean on dummies compresses the gaps by a square root.**

Where one-hot stops agreeing — a real reversal

```
20 rows, three nominal attributes.  Level frequencies:
  f1: {'a': 10, 'b': 10}    f2: {'P': 10, 'Q': 10}    f3: {'u': 19, 'w': 1}
column standard deviations (sqrt(pi(1-pi)) for a dummy):
  0.5000  0.5000  0.5000  0.5000  0.2179  0.2179

obj  f1   f2   f3   | mismatches |      (p-m)/p |     z-dummy L2
x0   a    P    u    |          0 |     0.000000 |       0.000000
xA   a    P    w    |          1 |     0.333333 |       6.488857
xB   b    Q    u    |          2 |     0.666667 |       4.000000
```

Ranking flip

The mismatch ratio says `xA` (one mismatch) is nearer to `x0`. Standardised dummies say `xB` (two mismatches) is nearer. Nothing in the data changed — only the *frequency* of the level `'w'`, which occurs once in twenty. **A rare category buys influence the mismatch ratio never granted it.**

Ordinal: categories with an order

$$\text{rank } r \in \{1, \dots, M\} \quad \longrightarrow \quad z = \frac{r-1}{M-1} \in [0,1] \quad \longrightarrow \quad \text{treat as numeric}$$

Two scales, and a pair

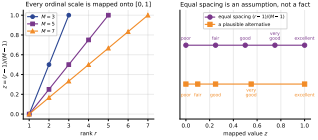
$M = 5$: poor 0, fair 0.25, good 0.5, very good 0.75, excellent 1. $M = 3$: junior 0, mid 0.5, senior 1.

P = (excellent, senior) $\to (1.00, 1.00)$; Q = (good, junior) $\to (0.50, 0.00)$.

$|1.00 - 0.50| = 0.50$, $|1.00 - 0.00| = 1.00$; Euclidean $\sqrt{1.25} = 1.118034$; mean absolute gap 0.750000.

Why divide by $M - 1$

So scales of different length are commensurable. A three-level and a seven-level attribute both span exactly $[0, 1]$, and neither dominates the sum by accident.



Equal spacing says poor-vs-fair and very-good-vs-excellent are the same distance apart. **That is an assumption you should be willing to defend.**

Gower: one weighted average across the types

$$d(i,j) = \frac{\sum_{f=1}^{p} \delta_f^{(ij)} \, d_f^{(ij)}}{\sum_{f=1}^{p} \delta_f^{(ij)}}$$

$d_f^{(ij)}$ — one number per attribute, in $[0, 1]$

numeric: $|x_{if} - x_{jf}| / R_f$, R_f the range
nominal / symmetric binary: 0 if equal, else 1
asymmetric binary: 0 if equal, else 1
ordinal: map to z, then $|z_{if} - z_{jf}|$

$\delta_f^{(ij)}$ — the indicator, and it does work

$\delta_f = 0$ when the attribute is missing for either object, **or** when the attribute is asymmetric binary and both objects are 0. Otherwise $\delta_f = 1$.

A dropped attribute leaves *both* the numerator and the denominator.

By hand: four applicants, four attribute types

id	age	exp	degree	city	car	python	patent
A	24	2	BTech	Dehradun	1	1	0
B	29	6	MTech	Dehradun	0	1	0
C	41	18	PhD	Mumbai	1	0	1
D	26	3	BTech	Delhi	0	1	0

$d(A, B)$, attribute by attribute

Ranges: $R_{\text{age}} = 41 - 24 = 17$, $R_{\text{exp}} = 18 - 2 = 16$. Ordinal: BTech 0, MTech 0.5, PhD 1.

$$\underbrace{\tfrac{5}{17}}_{0.294118} + \underbrace{\tfrac{4}{16}}_{0.250000} + \underbrace{|0 - 0.5|}_{0.500000} + \underbrace{0}_{\text{same city}} + \underbrace{1}_{\text{car 1 vs 0}} + \underbrace{0}_{\text{python 1,1}} + \underbrace{\text{skipped}}_{\text{patent 0,0}} = 2.044118$$

$$d(A, B) = 2.044118 \,/\, 6 = \mathbf{0.340686}$$

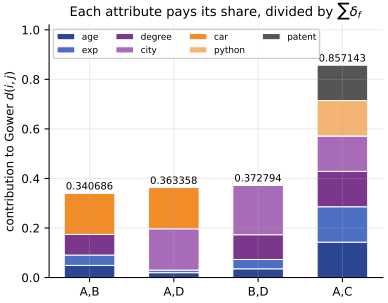
Where the asymmetric rule bites: $d(B, D)$

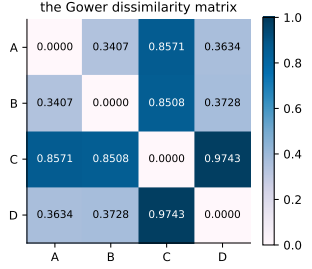
attribute f	delta_f	d_f	delta*d
age (numeric)	1	0.176471	0.176471
experience (numeric)	1	0.187500	0.187500
degree (ordinal)	1	0.500000	0.500000
city (nominal)	1	1.000000	1.000000
car (asym binary)	0	0.000000	0.000000
python (asym binary)	1	0.000000	0.000000
patent (asym binary)	0	0.000000	0.000000
SUM	5		1.863971

d = 1.863971 / 5 = 0.372794

Two attributes were dropped: neither B nor D owns a car, neither holds a patent. The denominator is 5, not 7. Divide by 7 and you get 0.266282 — a different answer and a wrong one.

Drawn: the contributions and the matrix





Nearest to A: **B** 0.340686 then **D** 0.363358 then C 0.857143. B and D are separated by 0.022672 — one attribute either way would swap them, which is why the weighting is worth arguing about.

Drop the type handling and the answer changes

```
E = [[age, exp, DEGREE.index(deg), cities.index(city), car, py, pat] ...]
Draw = squareform(pdist(E, "euclidean"))
```

```
nearest to A, raw Euclidean : D
nearest to A, z-scored      : D
nearest to A, Gower         : B
```

What label-encoding quietly asserted

That `city` is a number on which Delhi (0) $<$ Dehradun (1) $<$ Mumbai (2), so Delhi is "closer to" Dehradun than to Mumbai by exactly one unit. Nobody chose that ordering, the data does not support it, and it changed which applicant is A's nearest neighbour.

`LabelEncoder` on a nominal column is the single commonest silent bug in student clustering code.

Predict first #4 — who is row 1's neighbour?

Ten customers. Two attributes: annual fee (in rupees, around 50 000) and a usage ratio in $[0, 1]$. Query is row 1: fee 50 000, usage 0.50.

Row 2 is (50 300, 0.90). Row 4 is (49 500, 0.42).

Which one is nearer? Now: does your answer survive standardising?

Predict first #4 — who is row 1's neighbour?

Ten customers. Two attributes: annual fee (in rupees, around 50 000) and a usage ratio in $[0, 1]$. Query is row 1: fee 50 000, usage 0.50.

Row 2 is (50 300, 0.90). Row 4 is (49 500, 0.42).

Which one is nearer? Now: does your answer survive standardising?

```
 row          fee    usage |        d raw   d z-scored
   1        50000     0.50 |     0.000000     0.000000   <- query
   2        50300     0.90 |   300.000267     1.823190
   4        49500     0.42 |   500.000006     0.467002
nearest neighbour, raw      : row 2  (d = 300.000267)
nearest neighbour, z-scored : row 4  (d = 0.467002)
```

Predict first #4 — who is row 1's neighbour?

Ten customers. Two attributes: annual fee (in rupees, around 50 000) and a usage ratio in $[0, 1]$. Query is row 1: fee 50 000, usage 0.50.

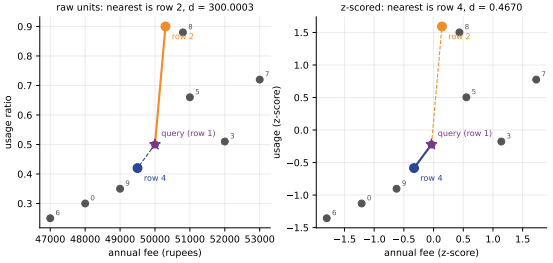
Row 2 is (50 300, 0.90). Row 4 is (49 500, 0.42).

Which one is nearer? Now: does your answer survive standardising?

```
 row          fee    usage |        d raw   d z-scored
   1        50000     0.50 |     0.000000     0.000000   <- query
   2        50300     0.90 |   300.000267     1.823190
   4        49500     0.42 |   500.000006     0.467002
nearest neighbour, raw      : row 2  (d = 300.000267)
nearest neighbour, z-scored : row 4  (d = 0.467002)
```

Different row. And the share of raw squared distance carried by the fee column, over all nine other rows: mean **1.000000** to six decimal places. The usage column was not consulted.

Drawn: the same ten points, twice — and on real data




```
wine: 178 rows, 13 features
rows whose nearest neighbour CHANGES IDENTITY under z-scoring: 168 / 178 (94.4%)
one column, proline (variance 98609.6), carries 0.997738 of the total
raw squared distance.
```

An unscaled distance is a distance in whichever column has the biggest numbers.

By hand: z-score against mean absolute deviation

$x = 1, 2, 3, 4, 100$. Mean $= 22$.

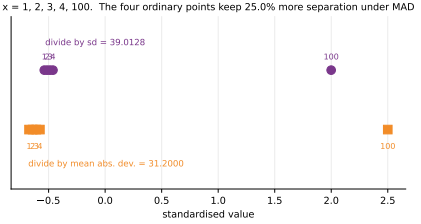
$\sigma = \sqrt{\frac{21^2+20^2+19^2+18^2+78^2}{5}} = \sqrt{\frac{7610}{5}} = \sqrt{1522} = 39.012818$

$s = \frac{21+20+19+18+78}{5} = \frac{156}{5} = 31.200000$ (the **mean absolute deviation**)

```
gap between x=1 and x=2 after scaling:
  by sd  0.025633       by mad  0.032051       ratio 1.2504
the outlier's own score:  sd 1.9993   mad 2.5000
```

σ is inflated by the *square* of the outlier's deviation; the mean absolute deviation only linearly. So σ crushes the four ordinary points together, and MAD scaling leaves them **25.0**% further apart from each other.

Drawn — and reported honestly



dataset	column	sd	mad	sd/mad
breast_cancer	area error	45.4510	27.1965	1.6712
wine	od280/od315_of_diluted_wines	0.7080	0.6117	1.1573
iris	petal length (cm)	1.7594	1.5627	1.1258

For a Gaussian column $\sigma/s \to \sqrt{\pi/2} = 1.2533$. Every `wine` and `iris` column sits near that, so there the two choices barely differ. **MAD scaling earns its keep when the tail is real** — as on the `breast_cancer` error columns.

Three ways to name "the middle"

Centroid

The componentwise mean $\bar{c} = \frac{1}{n}\sum_i x_i$.

Minimises $\sum_i \|x_i - c\|^2$.

Usually **not** a data point.

Needs numeric attributes.

Medoid

The object with the smallest total distance to the rest.

Is a data point. Works with any dissimilarity matrix, including Gower.

Mode

The most frequent value per attribute.

For purely nominal data, where means are meaningless ("the average colour").

This distinction is the whole difference between k-means and k-medoids, and it is why k-medoids can cluster mixed-type data and k-means cannot.

By hand: a three-point cluster

$A = \{(1,1), (2,1), (1,2)\}$

Centroid $= \left(\frac{1+2+1}{3}, \frac{1+1+2}{3}\right) = (1.333333,\ 1.333333)$ — not one of the three points.

Medoid — total distance to the others:

point	to the others		total
$(1,1)$	1.000000	1.000000	**2.000000**
$(2,1)$	1.000000	1.414214	2.414214
$(1,2)$	1.000000	1.414214	2.414214

Medoid $= (1,1)$ — and it *is* one of the three points.

The identity that makes the mean the right centre

$$\sum_i \|x_i - \bar{c}\|^2 = \frac{1}{2n} \sum_i \sum_j \|x_i - x_j\|^2$$

```
SSE(A) = sum_i ||x_i - centroid||^2          = 1.333333
       = (1/2n) sum_i sum_j ||x_i - x_j||^2  = 1.333333
difference 0.00e+00
  best grid point (1.3330, 1.3330), SSE 1.333334
```

The right-hand side does not mention $\bar{c}$ at all, so the within-cluster scatter is a property of the *pairwise distances*. A brute-force search over a fine grid finds the minimum exactly where the mean is — which is **why** k**-means uses means**.

By hand: distance between two clusters

$A = \{(1,1),(2,1),(1,2)\}$
$B = \{(5,5),(6,5),(5,6)\}$

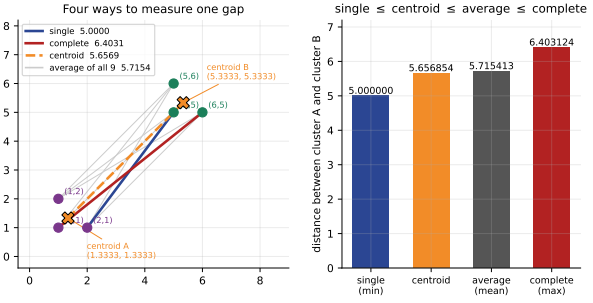
Nine cross distances, all computed with Pythagoras.

	b1=(5,5)	b2=(6,5)	b3=(5,6)
a1=(1,1)	5.656854	6.403124	6.403124
a2=(2,1)	5.000000	5.656854	5.830952
a3=(1,2)	5.000000	5.830952	5.656854

The four classical linkages

single (min) $= 5.000000$ **complete** (max) $= 6.403124$ **average** (mean of all nine) $= 5.715413$ **centroid** $= \|(1.3333, 1.3333) - (5.3333, 5.3333)\| = 4\sqrt{2} = 5.656854$

Drawn: four ways to measure one gap



Ordering: single 5.0000 $\leq$ centroid 5.6569 $\leq$ average 5.7154 $\leq$ complete 6.4031. Centroid $\leq$ average always: the distance between the means is at most the mean of the distances (Jensen). Gap here 0.058559.

Ward: the fifth one, and it is not a distance

```
SSE(A) 1.333333 + SSE(B) 1.333333 = 2.666667
SSE(A union B)                    = 50.666667
increase on merging               = 48.000000
closed form (nA nB/(nA+nB)) ||cA-cB||^2 = 48.000000   diff 1.42e-14
```

What Ward measures

Not how far apart the clusters are, but **how much within-cluster scatter merging them would create**. Here 48.000000.

Which is why

`scipy`'s `ward` reports 9.797959 where the other four report numbers near 5.7. Do **not** compare a Ward height with a single-linkage height. Different units.

`scipy.cluster.hierarchy.linkage` reproduces all four hand numbers exactly: 5.000000, 6.403124, 5.715413, 5.656854.

Predict first #5 — one algorithm, eight distances

`AgglomerativeClustering(n_clusters=3, linkage='average')` on `load_wine`, scored against the true cultivar with the adjusted Rand index.

Rank these: Euclidean, Manhattan, Chebyshev, cosine — raw and z-scored. Which combination wins?

Predict first #5 — one algorithm, eight distances

`AgglomerativeClustering(n_clusters=3, linkage='average')` on `load_wine`, scored against the true cultivar with the adjusted Rand index.

Rank these: Euclidean, Manhattan, Chebyshev, cosine — raw and z-scored. Which combination wins?

metric	ARI raw	ARI z-scored
euclidean	0.2926	-0.0054
manhattan	0.3186	0.4730
chebyshev	0.2926	-0.0068
cosine	0.0535	0.7847
kmeans (L2sq)	0.3711	0.8975

Predict first #5 — one algorithm, eight distances

`AgglomerativeClustering(n_clusters=3, linkage='average')` on `load_wine`, scored against the true cultivar with the adjusted Rand index.

Rank these: Euclidean, Manhattan, Chebyshev, cosine — raw and z-scored. Which combination wins?

metric	ARI raw	ARI z-scored
euclidean	0.2926	-0.0054
manhattan	0.3186	0.4730
chebyshev	0.2926	-0.0068
cosine	0.0535	0.7847
kmeans (L2sq)	0.3711	0.8975

From **0.0535** to **0.8975** — and the algorithm never changed. Only the distance did.

Two honest footnotes on that table

Standardising made one row *worse*

Average-linkage Euclidean fell from 0.2926 to −0.0054.

```
raw        sizes [130, 42, 6]
z-scored   sizes [174, 1, 3]
```

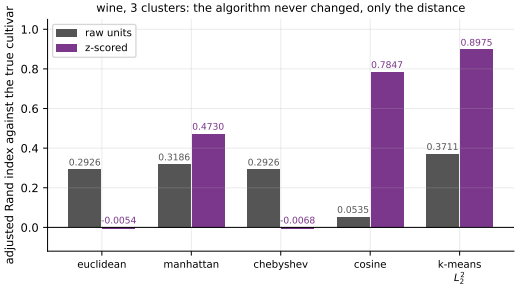
Not a distance failure: average linkage peeled off outliers once standardising let the small-scale columns speak. **Choose the distance and the linkage together.**

Cosine on raw wine did badly — correctly

Raw wine's vector length *is* proline, and proline separates the cultivars. Throwing the length away throws the signal away (0.0535). On text, where length is document length, throwing it away is exactly right.

And Euclidean on L_2-normalised rows gives the identical partition: ARI 0.0535 both ways.

Drawn: the whole argument in one chart



Ten bars. One algorithm. The spread from -0.0068 to 0.8975 is entirely the distance and the scaling. **Nothing you do to the algorithm will recover a badly chosen distance.**

Summary — twelve lines (1 of 2)

1. A dissimilarity is a **metric** if it satisfies non-negativity, identity, symmetry and the triangle inequality. **Squared Euclidean** and **cosine distance** fail the triangle inequality; **KL divergence** fails symmetry.
2. $d_p = (\sum_f |x_f - y_f|^p)^{1/p}$: p=1 Manhattan, p=2 Euclidean, $p \to \infty$ Chebyshev. For $(1, 2)$ and $(4, 6)$: **7**, **5**, 4.497941, **4**.
3. $d_1 \geq d_2 \geq \cdots \geq d_\infty$ always. Large p punishes a single bad attribute; p=1 does not. On q=$(0, 0)$, u=$(3, 3)$, v=$(4.5, 0)$ they pick different neighbours.
4. Cosine measures direction and discards length. A=$(1, 2)$, B=$10A$, C=$(2, 1)$: Euclidean says C (1.4142 vs 20.1246), cosine says B (1.0000 vs 0.8000).
5. $\|\hat{a} - \hat{b}\|^2 = 2(1 - \cos)$, so cosine *is* Euclidean on L_2-normalised rows.

Summary — twelve lines (2 of 2)

6. Binary: build q, r, s, t. **Simple matching** $(q+t)/p$ for symmetric attributes; **Jaccard** $q/(q+r+s)$ for asymmetric. The question is whether a shared *absence* is evidence.
7. They can rank two pairs in **opposite orders**: 0.8000 vs 0.6000 against 0.3333 vs 0.6000. Padding with shared zeros drives simple matching to 0.9980 and leaves Jaccard at 0.3333.
8. Nominal: $(p - m)/p$. One-hot with L_1 is *exactly* the same measure; with L_2 the ranking survives but the gaps compress by a square root; with *standardised* dummies the ranking can flip.
9. Ordinal: $z = (r - 1)/(M - 1)$, then treat as numeric. Equal spacing is an assumption.
10. Mixed: Gower, $\sum \delta_f d_f / \sum \delta_f$. A 0–0 pair on an asymmetric binary attribute sets $\delta_f = 0$ and leaves both sums. $d(A, B) = \mathbf{0.340686}$, $d(B, D) = \mathbf{0.372794}$.
11. Standardisation changes which object is nearest — **94.4**% of `wine` rows. MAD scaling is the robust alternative; $\sigma/s \to \sqrt{\pi/2} = 1.2533$ for a Gaussian column.
12. Centroid $\neq$ medoid. Cluster distances: single 5.0000, centroid 5.6569, average 5.7154, complete 6.4031 — in that order, always.

Before the next lecture

Read and do

`notes.pdf` — **ten practice questions with full worked solutions**, four of them pure hand computation. Rebuild the three-patient table with a pen, then $d(A, C)$ and $d(C, D)$ for the applicant table showing every δ_f.

The habit to take away

Before you cluster anything, write down for each column: its **type**, whether it is **symmetric**, and its **units**.

What the matrix costs

n	pairs	MB at 8 bytes
150	11175	0.1
1000	499500	4.0
10000	49995000	400.0
100000	4999950000	39999.6

At a hundred thousand objects the dissimilarity matrix alone is **40 GB**. This is why the hierarchical methods of the next lecture are quadratic in memory, and why k-means — which never forms the matrix — scales further.

Materials: `https://github.com/tali7c/Applied-Machine-Learning`

Sources: Han, Kamber and Pei, *Data Mining: Concepts and Techniques* (3e), Ch. 10.;James et al., *An Introduction to Statistical Learning with Python*, Ch. 12.;Müller and Guido, *Introduction to Machine Learning with Python*, §3.5.;Ester, Kriegel, Sander and Xu, "A density-based algorithm for discovering clusters (DBSCAN)", KDD 1996.